本章路線

這章要回答的核心問題是：如何在隔離多個 guest 的同時，保持指令語意、裝置介面與資源上限？

前置知識。 OS 結構、CPU trap、位址轉換與 I/O。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

學習目標

- 區分 type 1/type 2 hypervisor
- 分析 trap-and-emulate 與 binary translation
- 說明 memory/I/O 虛擬化的安全與成本

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

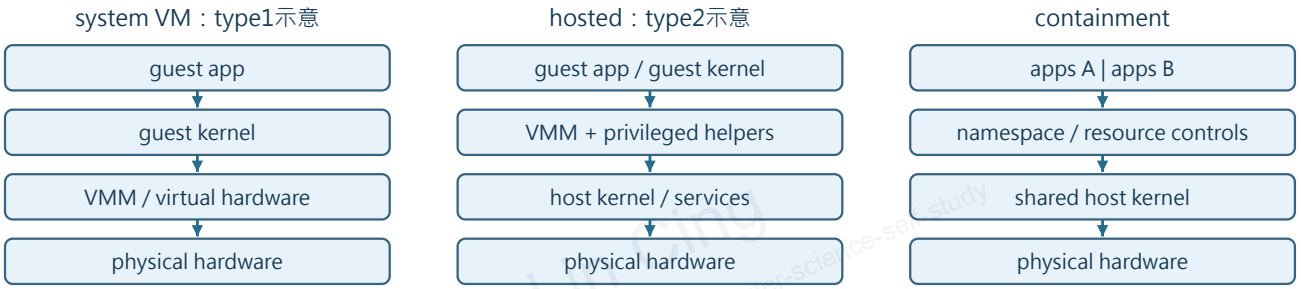
16.1 Overview

- system VM提供guest OS的CPU/memory/device介面。
- vCPU保存狀態，pCPU才實際執行。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

VM、host、guest 與 VMM 的邊界

控制資源的層不同：不能把 VM 與共用 kernel 的 container 混同



system VM向guest OS提供CPU/memory/device介面；VMM/hypervisor管理實體資源並實現虛擬介面。guest內的application仍呼叫guest OS，guest OS再控制虛擬device/CPU；不是所有guest syscall都必須立即進host kernel。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

VM、host、guest 與 VMM 的邊界 （續）

vCPU保存guest可見的register、PC、狀態與虛擬interrupt等，排到pCPU才實際執行。兩個VM都可見「physical page5」，卻由各自第二階段映射到不同host frame；同名不等於同一資源。VMM必須管理共享CPU、記憶體、裝置與管理介面的隔離。

system VM、process VM（如JVM）、cross-ISA emulator、OS-level containment是不同抽象。分類應問「對上提供哪個介面、誰持kernel、誰控制physical resource」，不只背type號碼。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.2 History

- VM/370以minidisk分割共享資源。
- fidelity/resource control/efficiency為不同目標。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

歷史動機與三個設計目標

IBM VM/370（1970年代）的minidisk把實體disk的不同區段呈現為guest獨立disk，讓多個OS共享一台mainframe。此種抽象需要保存地址範圍與隔離，而不是把完整實體disk register任意交給所有guest。

fidelity：除了明定差異，guest可見行為符合虛擬機器規格；resource control：guest不能奪走VMM控制的資源；efficiency：大部分可安全執行的工作可直接執行。效能目標不表示每種工作負載固定只慢某個百分比，也不表示時間觀察完全相同。

對傳統trap-and-emulate的常見充分條件是需要VMM仲介的sensitive instructions能可靠被攔截；失去此條件可改用binary translation/paravirtualization/硬體輔助。完整emulation可在軟體中解釋guest ISA，不需要CPU先有專用VM指令；專用硬體主要改可用技術與效率界線。

16.3 Benefits and features

- 整合server仍受同時峰值與共同故障限制。
- snapshot/clone/backup需要不同一致性與故障邊界。

Isolation、consolidation 與共同失效

VM隔離方便OS開發、測試多版本與整合低利用率server；若hypervisor/device emulator有漏洞，或多VM共用同一故障host，仍可能共同失效。資源隔離也須有CPU/memory/I/O限額，不能由「地址分開」推出鄰居不會拖慢自己。

原創capacity例：host8cores，VMM預留1core，A/B/C的峰值需求分別3/3/2cores；總8>剩餘7，即使平時平均各只用1，也不能保證三者同時峰值不排隊。overcommit依賴工作負載與服務目標，並非免費增加運算能力。

VM網路/共享磁碟可解決guest間分享，但重建了新的信任邊界。管理平面能clone/snapshot/讀disk image，本身是重要TCB，不因guest彼此隔離就可忽略管理者權限。

Suspend、snapshot、clone 與 backup 不同

suspend保存可恢復CPU/RAM/device狀態；disk snapshot保存磁碟某點的block版本；clone建立可獨立運行的副本。只copy正在寫入的多個disk files不一定得到一致時間點，guest RAM與disk也可能不匹配；需停機/一致snapshot或guest application quiescence等契約。

crash-consistent snapshot相當於某時刻斷電後可恢復的disk state，application-consistent還需應用的flush/transaction協調。外部mail/payment/database effects不在同一snapshot內，回滾VM不會自動撤回它們；clone也須處理重複IP/MAC/machine identity與credentials。

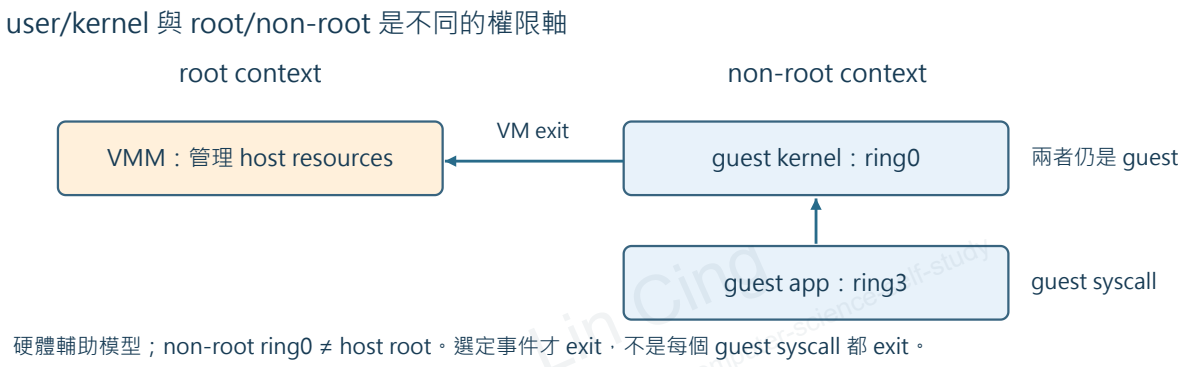
snapshot若與source disk位於同一故障範圍，不等於獨立backup。template提高建立一致性，但template中的漏洞/帳密也會被複製；cloud/VDI把資源建立變成可管理API，仍受同樣的隔離與外部狀態限制。

16.4 Building blocks

- trap、translation、hypercall與硬體輔助各有前提。
- shadow與two-stage分開推導地址/權限與walk成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trap-and-emulate：虛擬模式與實體模式分開



傳統無額外VM mode的模型：VMM持physical privileged mode，guest kernel與user都在較低實體權限執行，但VMM保留guest虛擬mode。guest kernel的特權操作trap，由VMM驗證guest原本是否有虛擬權，再emulate；guest user違法操作應得到guest exception，不能因到了VMM就被授權。

Trap-and-emulate：虛擬模式與實體模式分開 （續）

例guest kernel要求關interrupt：VMM把vCPU.virtualIF設0，延後此guest可遮罩interrupt的注入；不應讓它永久關掉整台host服務其他VM的真實interrupt。VMM保留host控制權，並保存guest可見狀態；之後guest重新開interrupt才按規格注入pending事件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不會 trap 的 sensitive instruction 與 translation

若某instruction在低權執行時靜默忽略部分狀態改變，guest kernel以為已修改虛擬flags，VMM卻未得到控制，單靠trap-and-emulate就失真。x86歷史POPF的特定flag/權限行為是這種問題的例子；不能由「未trap」推「對guest無害」。

binary translation讀guest instruction blocks，將需仲介的操作替換為等價、安全的VMM操作，其餘轉成可直接執行的host code，並cache翻譯結果。需要處理indirect branches、自修改code、page權限變更與exception-visible狀態；修改原code後不能沿用語義已變的translation cache。

paravirtualization另讓合作guest呼叫hypercall取代不易虛擬化操作；它與binary translation可解決相似問題，但對guest ABI/修改要求不同。硬體輔助也不表示device emulation、scheduler或I/O成本都消失。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hardware-assisted guest execution

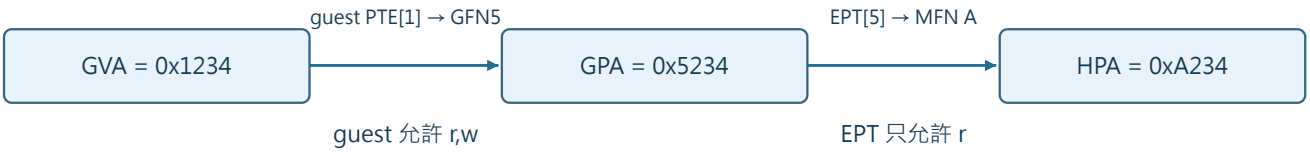
root/non-root或host/guest mode提供比傳統user/kernel更多的控制維度。guest OS可在non-root ring0，guest app在non-root ring3；VMM以root context管理。VM entry載入guest state；特定event產生VM exit，保存guest狀態、提供exit reason，由VMM處理再resume。

哪些instruction/event exit由架構與controls決定，不是每條guest instruction都exit，也不是每次guest syscall都exit。VMM要正確設定intercepts、保存狀態與事件注入，才能同時滿足fidelity/resource control。

DMA的IOMMU與interrupt remapping解決device端隔離/路由，與CPU instruction virtualization不同；不應把CPU page table或EPT誤當作自動約束所有device DMA的機制。

Shadow page table 與硬體兩階段轉譯

地址與權限逐層組合；不是只串兩個名詞



page size 4096 B；offset 0x234 在兩階段都保留。read 通過；write 第二階段拒。

Shadow table 可直接組合 VPN1 → MFN A，但需同步 guest PTE 更新與 translation invalidation。

guest page table：GVA→GPA；VMM第二階段：GPA→HPA。**hardware two-stage**（如EPT/NPT）由硬體在page walk中組合兩層；**shadow table**則由VMM維護GVA→HPA的組合映射，讓一般MMU直接使用shadow，並攔截/追蹤guest page-table更新以保持一致。兩者不能只因都有額外表就混成同一演算法。

Shadow page table 與硬體兩階段轉譯 （續）

本例page4096 B，GVA0x1234=(VPN1,offset0x234)，guestPTE[1]=(GFN5,rw)，GPA=0x5234；EPT[5]=(MFN0xA,r)，HPA=0xA234。read需兩層都允許，write被EPT拒；guest以為可寫不代表有權寫host page。若guestPTE本身deny，應按guest paging規格處理guest fault；第二階段違規由VMM的相應機制處理，不可一律當同一fault。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

無cache的 nested walk：為何不只「兩倍」

假設guest m-level、第二階段n-level，沒有TLB/page-walk cache/large pages。每讀一個guest PTE的GPA，先需n次第二階段PTE讀取，再讀guest PTE本身，因此m(n+1)次；最後的data GPA再需n次第二階段讀取。共m(n+1)+n=(m+1)(n+1)−1次**table references**，再加一次data存取。

m=n=4：4×5+4=24次table reads，加data=25；不是4+4=8，也不是所有load永遠25次。TLB命中可避walk，page-walk cache/huge pages也可降低。shadow的單層walk較少，但guest PTE變更需VMM維護，是不同成本所在。

TLB/translation cache還需在guest/VMM改mapping或權限時按架構失效；切vCPU/VM要用正確address-space tagging或flush，不能把上一VM的相同GVA映射誤用到另一VM。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.5 VM types and implementations

- lifecycle、type0/1/2、PV、processVM、emulation與containment依抽象層比較。
- shared ring以counter、ownership與發布順序保存狀態。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

VM lifecycle 與 type0/1/2 的責任

建立VM先驗CPU、memory、disk、network需求與policy，再配置vCPU、地址空間與device state，載入guest；running可pause/resume、snapshot、遷移，delete前先停止device/事件並回收資源。只刪管理紀錄而留活DMA或網路身份不是正確銷毀。

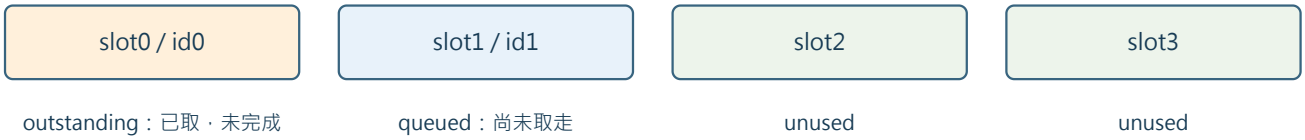
type0是firmware/hardware partition類型，可dedicate或分享部分資源；type1直接管理host資源，含某些general-purpose kernel中的VMM；type2依host OS提供process/device/memory服務。type2常仍有特權helper/driver與硬體輔助，不能一概說host完全不知虛擬化或永不能用VT-x。

分類不是對效能/功能的無條件排名；nesting是否支援由VMM、CPU與設定決定，不是邏輯上只有type0可嵌套。管理範圍、TCB與resource path才是可檢驗的差異。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Paravirtual I/O：descriptor、發布與回應

四槽順序完成模型：backend 取走不等於 frontend 可重用



outstanding：已取，未完成　queued：尚未取走　unused　unused

req_prod=2，req_cons=1，rsp_prod=0，rsp_cons=0；used=2，free=4−2=2。

完成 id0：rsp_prod→1；frontend 消耗 response 後 rsp_cons→1，才釋 slot 容量。

固定歷史參考為Xen SOSP2003的合作guest、hypercalls與shared-ring設計；下面是明確簡化的單producer/consumer模型，不冒充所有Xen版本的完整ABI。counter單調遞增，slot=index mod4；req_prod=2、req_cons=1、rsp_prod=0、rsp_cons=0，因此queued=1、outstanding=1、response=0、used=2、free=2。

Paravirtual I/O：descriptor、發布與回應 (續)

backend完成id0後rsp_prod=1，outstanding變0、response變1；frontend消耗此response後rsp_cons=1，used=req_prod−rsp_cons=1、free=3。不能在backend剛取走request時就重用descriptor slot，因為它可能仍等completion。

Shared ring 的同步與驗證

producer先填descriptor，再release發布req_prod；consumer以acquire讀到新prod後才可讀descriptor。response也需先寫結果後發布rsp_prod；frontend確認完成、取走response後，才推rsp_cons釋槽。event-channel/doorbell是通知，不取代counter predicate；重新檢查後再睡眠避免遺失喚醒。

不變量0≤req_prod−rsp_cons≤N，與rsp_cons≤rsp_prod≤req_cons≤req_prod（本簡化順序完成模型）保持四區間不重疊。backend仍須驗id、operation、block範圍、長度與buffer grant/ownership；shared memory不能讓未受信guest任意DMA到host。out-of-order completion需要另一個id/slot管理模型，不能直接套本順序counter假設。

Process VM、emulation 與 containment

JVM/.NET式process VM提供language bytecode/runtime介面，不是完整guest OS；cross-ISA emulation解釋/翻譯不同ISA的instruction與machine state，可能慢但能執行不相容二進位。兩者都不能直接用「是否有自己的hardware ring0」作唯一判準。

OS containment（書中Solaris10 zones、BSD jails等）共用host kernel，隔開namespace/resource/security context；不是每個container有獨立kernel。它能有不同userland/network view，但kernel bug的共用故障範圍與VM不同。Solaris10的global zone管理non-global zones，系統服務與resource controls需具體配置，不宣稱每個zone獨立實作整套kernel scheduler。

選擇抽象取決於需求：測試另一kernel需system VM或emulator；同kernel下隔開服務可用containment；執行bytecode需對應runtime。這是機制適配，不是單一方案永遠最佳。

16.6 Virtualization and OS components

- guest排thread而host排vCPU；CPU time不同於wall time。
- balloon/COW/IOMMU/storage/migration以可追蹤狀態推導。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CPU scheduling：guest 排thread，VMM 排vCPU

兩個pCPU；A0/A1與B0/B1共四vCPU，host overhead先忽略。0–10 ms排A0/A1，10–20排B0/B1，20–30再A，30–40再B。A某thread需20 ms CPU，從t0開始在A0於0–10與20–30執行，牆鐘t30才完；B對應thread在10–20與30–40執行，t40完。平均各得一半pCPU，不代表每個20 ms budget都剛好40 ms latency。

guest ready thread可能因其vCPU未獲pCPU而等待；guest內lock holder被deschedule時，另一vCPU spinning可能浪費其份額。virtual clock、wall clock與實際vCPU runtime要分開；virtual timer到期後也可能等vCPU才能處理。real-time保證需要host admission/隔離/排程條件，不能只在guest改priority。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Memory overcommit 與 balloon 的真正方向

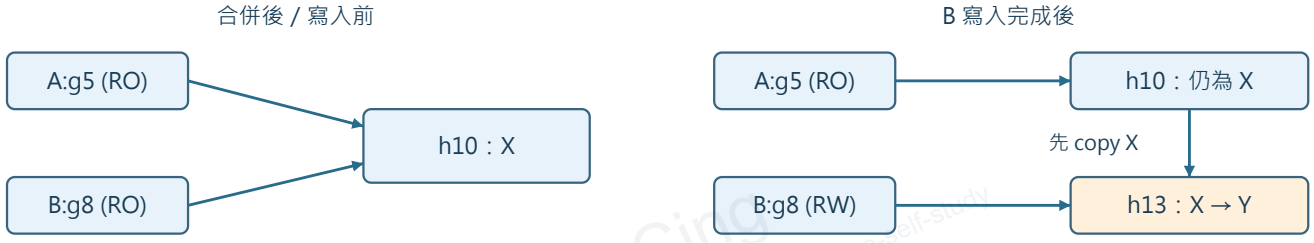
歷史參考固定Waldspurger的ESX Server OSDI2002設計。balloon driver在guest內配置/保留guest pages並報告其PFNs；VMM可回收對應host backing，使guest依自己策略選擇要釋放/換出的其他頁。inflate使guest可供工作負載使用的memory減少，不是增加host實體RAM。

原創容量例：host4096 MiB，VMM預留512，guest可用3584；A/B各要求2304 MiB常駐backing，總需求4608，尚未全部獲得，超出可供容量1024。令A balloon256、B balloon768，將需求降至target backing A2048+B1536=3584，與VMM512合4096。這是需求與配置目標的守恆，不是宣稱物理RAM曾同時裝下4608 MiB。guest nominal配置可不變，但不能讓其application無成本地繼續使用全部原需求頁。

balloon配置的頁不應再被guest工作負載使用；VMM仍須保護無backing映射，若guest錯誤觸碰則fault/重新back，不可直接碰到已交另一guest的host frame。deflate前重建可安全使用的backing。driver不合作或回收不夠快時，VMM可能需另用paging/限制，不能假定一定成功。

Page sharing、COW 與 double paging

先完整比對再共享；寫入前先分離（host-frame 層）



B 的 write fault：先配置 / copy → 改 B 映射與 translation 失效 → 再放行寫入；A 映射不變。

A:g5→h10、B:g8→h12，內容同為X。hash僅篩candidate，完整bytes比較一致後才合併至h10，兩邊有效write permission去掉、維持reference。B寫Y時fault，先copy X到新h13、B:g8改→h13並按規格失效舊translation，才讓B寫；A仍見X。不能先放行write再copy。

Page sharing、COW 與 double paging （續）

VMM swapping不知道guest哪些頁最重要；guest可能先把某頁寫virtual swap，VMM又把承載該guest頁的host frame換出，造成double paging與額外I/O。balloon讓guest選victim可改善資訊落差，但不是任何情況都比所有paging快。sharing掃描也有CPU/同步成本及隔離政策；本例不宣稱跨tenant sharing永遠啟用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

I/O 與 virtual networking 的三條路

emulated device：guest driver操作虛擬register，VMM/device model驗證並映到host driver；paravirtual device：guest與backend用合作queue；passthrough：device直接服務guest，仍需IOMMU/address isolation、interrupt路由與安全的reset/ownership transfer。

若guest DMA address0x5000映到host0xA000，IOMMU須只容許配置的範圍；guest CPU的EPT限制本身不等於DMA限制。passthrough減少部分仲介，可能讓snapshot/migration需要額外device state支援，不能只搬RAM就無條件遷移。

bridge讓VM接入相應L2網路；NAT由host把內部位址/port映到外部tuple。例A10.0.2.15:5000與B10.0.2.16:5000分別映host:40001/40002，可共用外部IP；所以「每個VM必須有不同公網IP」不成立。firewall規則仍要界定guest間及對外通道。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

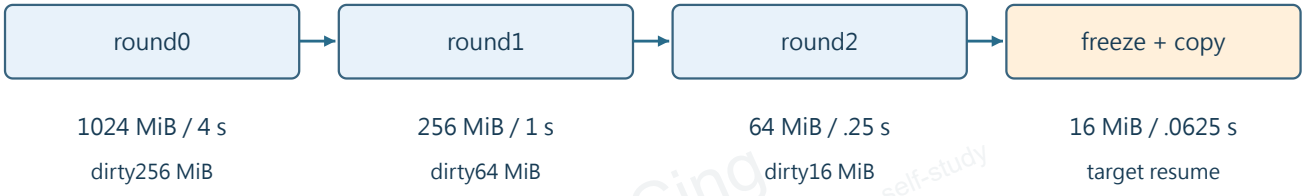
Storage image 與 snapshot 的 block 追蹤

guest disk的LBA只是虛擬disk位置，再由image mapping/host filesystem到實體儲存；也可能直接用block volume，不保證永遠只是一個普通file。thin provisioning只延後配置，不保證最大邏輯容量同時寫滿仍有空間。

原創snapshot chain：base存blocks{0:A,1:B,2:C}；建立snapshot後active delta寫{1:B′}。read1先查delta得B′，read2落base得C；回到snapshot選base可讀回B，但外部已寄出的mail不會撤回。clone若共享base，base不能在仍被引用時刪除；完整backup須包含必要chain/metadata與一致時間點。

Pre-copy migration：每一輪都能核算

正常路徑 pre-copy：總時間與 downtime 分開



B=256 MiB/s · d=64 MiB/s；總 1360 MiB / 5.3125 s · pause .0625 s（未含控制開銷）。

假設同VMM/兼容CPU、shared storage、無不可遷移device，網路有效copy B=256 MiB/s，guest每秒產生d=64 MiB不同需重送頁、忽略協定/CPUstate成本。R_0=1024 MiB，t_0=4 s，期間dirty R_1=256；下一輪t_1=1 s、R_2=64；下一輪t_2=.25 s、R_3=16。此時freeze，copy最後16需.0625 s；總copy1360 MiB、5.3125 s，其中pause .0625 s。

Pre-copy migration：每一輪都能核算 （續）

一般模型$R_{i+1}=(d/B)R_i$，若$d/B<1$幾何收斂；$d \geq B$時此模型不縮小，需限制輪數/降dirty rate/延長pause或改方法。真實dirty頁可重複、頻寬波動且RAM有限，本式是明確假設的模型，不是所有migration的精確預測。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stop-and-copy 與單一活動副本

freeze不只停user code，還需quiesce可改guest狀態的device/DMA與交付中的事件；保存vCPU/device/dirty RAM及必要network state，讓target取得一致狀態。轉移ownership前source不可再對外執行guest，target可運作後再清source資源。

若source只因沒收到ack便重新運行、target其實已啟動，就可能split brain。安全遷移協定須有明確commit/ownership與故障恢復規則，不能以「等target ack」代替所有failure case證明。本節簡化正常路徑只證pause時的consistent handoff，不宣稱任意network partition下仍透明可用。

網路需維持可達地址/MAC等對外身份與路由更新；TCP可容忍短停頓不表示pause=0。shared-storage是此pre-copy模型前提，其他系統也可做block migration，因此不能由本模型推出「任何live migration都不能搬disk」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.7 Examples

- 以原書Workstation架構和Xen2003/ESX2002歷史研究比較。
- SE7 JVM的bytecode與runtime不等於system VM。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Workstation 與 Xen／ESX 的歷史對照

Workstation採書中2013年前後的hosted架構示意：guest app→guest kernel→virtual device/虛擬CPU→VMM/host kernel服務→hardware；普通guest instruction可直接執行，I/O可能經host服務。此處不把示意稱為某release的完整程式碼或量測。

固定原始研究例：Xen SOSP2003以合作guest/hypercalls/shared ring減少不易虛擬化的介面；ESX OSDI2002以balloon/content sharing等管理overcommit。二者的介面與研究年代不同，不能把早期PV guest規則宣稱為所有後來Xen的唯一模式，也不能從paper某個benchmark泛化固定效能比例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

JVM：bytecode verifier 與執行引擎

以Java SE7時期的一般JVM抽象為歷史比較：class loader載入class，驗證型別、stack使用與合法控制流；execution engine可interpret或JIT；runtime提供heap/GC及安全機制。JVM不包含另一個通用guest kernel，仍依host OS取得memory、thread與I/O。

bytecode例`iconst_2; iconst_3; iadd; istore_1`：operand stack依序[]→[2]→[2,3]→[5]→[]，local1=5。若初始[]直接iadd，需兩個int卻沒有，verifier可拒；不是等硬體stack underflow才算錯。這不保證業務邏輯正確。

JIT不一定第一次呼叫就編完整method，可能依hotness/tiering；GC也不是單純把「不再使用」的語義自動判斷，而是依runtime的可達性/collector規則回收。語言規格與實作策略應分開。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

VM 的兩個必守條件

fidelity要求每個guest-visible state transition符合虛擬機器規格；resource control要求guest不能逃過VMM對CPU、地址與device的管制。trap/translation/hypercall/hardware assistance是達成方式，不是保證本身。

GVA→GPA→HPA的權限交集、shared ring發布/重用順序、COW寫入前分離、migration單一活動副本，都在保護「此guest以為自己的狀態」與「host實際持有的狀態」一致。單次demo成功不能代替所有轉移條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

分母與邊界的檢查

vCPU數不等於pCPU容量；nominal RAM不等於當下host backing；logical disk容量不等於已配置bytes；snapshot不是外部世界回滾；pre-copy總傳輸時間不是downtime。每一個數字都要標在哪一層。

練習以mode、地址、walk次數、ring四counter、balloon容量、COW與migration遞迴重建機制；計算前先標出guest、host與device各自的狀態與存取條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料

Silberschatz、Galvin、Gagne，*Operating System Concepts*，第九版，第16章，印刷頁711–738。原創圖例與數值模型重建全章議題；shadow與hardware two-stage分開定義。

Barham等，Xen and the Art of Virtualization，SOSP2003，§2–3：歷史PV介面/hypercall/shared I/O設計。

Waldspurger，ESX memory management，OSDI2002：Ballooning與Content-Based Page Sharing：核對guest分配/host回收、完整byte比較與COW。容量與migration數例均為明列假設的原創模型。

Java Virtual Machine Specification，Java SE7，§4.10：核對bytecode verifier的型別/operand stack規則；未假定所有JIT實作相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study